

Báo cáo bài tập EX17.1:

Lab1_Graph_embedding_Problem

Introduction



Welcome to this lab! At this lab, we will learn:

1. How to build a graph from a file or create a simple graph by ourself
2. Implement DeepWalk in the simplest way based on the paper [DeepWalk](#). You can implement your own code or study some packages already made. I recommend reading Karateclub framework, which is well-known for Unsupervised Learning on Graphs

Exercise

Download data and install packages

```
[1]: from pathlib import Path
import importlib.util
import os
import shutil
import subprocess
import sys

if importlib.util.find_spec("gdown") is None:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "gdown"])

import gdown

work_dir = Path.cwd()
archive_path = work_dir / "lab1.rar"

if not archive_path.exists():
    gdown.download(id="1vqsjGzGZnpCEgHliEsVmAvzm9_1h3L-Y", output=str(archive_path), quiet=False)

extractors = []
for name in ("unrar", "7z", "7zz", "7za", "tar"):
    path = shutil.which(name)
    if path:
        extractors.append((name, path))
```

```

extractors = []
for name in ("unrar", "7z", "7zz", "7za", "tar"):
    path = shutil.which(name)
    if path:
        extractors.append((name, path))

for candidate in (
    Path("C:/Program Files/7-Zip/7z.exe"),
    Path("C:/Program Files (x86)/7-Zip/7z.exe"),
    Path("C:/Program Files/WinRAR/UnRAR.exe"),
    Path("C:/Program Files/WinRAR/WinRAR.exe"),
):
    if candidate.exists():
        extractors.append((candidate.stem.lower(), str(candidate)))

for root in (Path(os.environ.get("ProgramFiles", "")), Path(os.environ.get("ProgramFiles(x86)", ""))):
    if root.exists():
        try:
            found_7z = next(root.rglob("7z.exe"), None)
            if found_7z:
                extractors.append(("7z", str(found_7z)))
        except OSError:
            pass

```

```

last_error = None
for kind, exe in extractors:
    try:
        if kind.startswith("unrar") or kind == "winrar":
            cmd = [exe, "x", "-y", "-o+", str(archive_path), str(work_dir) + os.sep]
        elif kind == "tar":
            cmd = [exe, "-xf", str(archive_path), "-C", str(work_dir)]
        else:
            cmd = [exe, "x", "-y", f"-o{work_dir}", str(archive_path)]
        subprocess.check_call(cmd)
        break
    except Exception as exc:
        last_error = exc
else:
    raise RuntimeError(
        "Cannot extract lab1.rar. Install 7-Zip or WinRAR, then run this cell again."
    ) from last_error

print("Ready:", work_dir / "lab1_small_edgelist.txt", work_dir / "lab1_big_edgelist.txt")

```

Ready: C:\Users\ADMIN\OneDrive\Máy tính\GR1-bai_tap_thay_linh\ML-exercises\Ex17\lab1_small_edgelist.txt C:\Users\ADMIN\OneDrive\Máy tính\GR1-bai_tap_thay_linh\ML-exercises\Ex17\lab1_big_edgelist.txt

Build a graph

```
[2]: import networkx as nx
import numpy as np
import torch
```

▼ TO DO: Create graphs

Create a graph from the file: "lab1_small_edgelist.txt"

```
[3]: # TO DO: Create a graph from the file: "lab1_small_edgelist.txt"
G_small = nx.read_edgelist(
    'lab1_small_edgelist.txt',
    nodetype=int,
    create_using=nx.Graph()
)
print('Nodes:', G_small.number_of_nodes(), '| Edges:', G_small.number_of_edges())
G_small
```

Nodes: 10 | Edges: 28

```
[3]: <networkx.classes.graph.Graph at 0x1e793c830e0>
```

TO DO: Create graphs

Create a graph from the file: "lab1_small_edgelist.txt"

```
[3]: # TO DO: Create a graph from the file: "lab1_small_edgelist.txt"
G_small = nx.read_edgelist(
    'lab1_small_edgelist.txt',
    nodetype=int,
    create_using=nx.Graph()
)
print('Nodes:', G_small.number_of_nodes(), '| Edges:', G_small.number_of_edges())
G_small
```

Nodes: 10 | Edges: 28

```
[3]: <networkx.classes.graph.Graph at 0x1e793c830e0>
```

Create a graph by generating nodes and edges.

```
[4]: # TO DO: Create a graph by generating nodes and edges.
G_gen = nx.gnm_random_graph(n=30, m=60, seed=42)
print('Nodes:', G_gen.number_of_nodes(), '| Edges:', G_gen.number_of_edges())
G_gen
```

Nodes: 30 | Edges: 60

```
[4]: <networkx.classes.graph.Graph at 0x1e796705950>
```

Implement DeepWalk

Packages: Import necessary packages

```
[5]: import networkx as nx
      from joblib import Parallel, delayed
      import random
      import itertools
      import numpy as np
      from gensim.models import Word2Vec
```

Utils: Processing data

```
[6]: def partition_num(num, workers):
      if num % workers == 0:
          return [num//workers]*workers
      else:
          return [num//workers]*workers + [num % workers]
```

TO DO: Implement DeepWalk

Algorithm 1 DEEPWALK(G, w, d, γ, t)

Input: graph $G(V, E)$

 window size w

 embedding size d

 walks per vertex γ

 walk length t

Output: matrix of vertex representations $\Phi \in \mathbb{R}^{|V| \times d}$

1: Initialization: Sample Φ from $\mathcal{U}^{|V| \times d}$

2: Build a binary Tree T from V

3: **for** $i = 0$ to γ **do**

4: $\mathcal{O} = \text{Shuffle}(V)$

5: **for each** $v_i \in \mathcal{O}$ **do**

6: $\mathcal{W}_{v_i} = \text{RandomWalk}(G, v_i, t)$

7: SkipGram($\Phi, \mathcal{W}_{v_i}, w$)

8: **end for**

9: **end for**

```
[7]: # Please implement your code here.
# 1. Try to build your own code
# 2. Try to use class RandomWalk built in the KarateClub framework (used in homework's solution)
class RandomWalker:
    def __init__(self, G, num_walks, walk_length):
        """
        :param G: Graph
        :param num_walks: a number of walks per vertex
        :param walk_length: Length of a walk. Each walk is considered as a sentence
        """
        self.G = G
        self.num_walks = num_walks
        self.walk_length = walk_length

    def _random_walk(self, start_node):
        walk = [start_node]
        while len(walk) < self.walk_length:
            cur = walk[-1]
            neighbors = list(self.G.neighbors(cur))
            if not neighbors:
                break
            walk.append(random.choice(neighbors))
        return [str(node) for node in walk]
```

```

# INFORMATION EXTRACTOR
def simulate_walks(self):
    # TO DO: Create walks
    walks = []
    nodes = list(self.G.nodes())
    for _ in range(self.num_walks):
        random.shuffle(nodes)
        for node in nodes:
            walks.append(self._random_walk(node))
    return walks

```

```

[8]: class DeepWalk:
    def __init__(self, graph, walk_length, num_walks):

        self.graph = graph
        self.w2v_model = None
        self._embeddings = {}

        self.walker = RandomWalker(graph, num_walks=num_walks, walk_length=walk_length)
        self.walks = self.walker.simulate_walks()

    def train(self, embed_size=128, window_size=5, workers=1, iter=5, **kwargs):

        kwargs["sentences"] = self.walks
        kwargs["min_count"] = kwargs.get("min_count", 0)
        kwargs["vector_size"] = embed_size # Changed from 'size' to 'vector_size' for gensim 4.0+
        kwargs["sg"] = 1 # skip gram
        kwargs["hs"] = 1 # deepwalk use Hierarchical Softmax
        kwargs["workers"] = workers
        kwargs["window"] = window_size
        kwargs["epochs"] = iter # Changed from 'iter' to 'epochs' for gensim 4.0+

        print("Learning embedding vectors...")
        model = Word2Vec(**kwargs) # Pay attention here
        print("Learning embedding vectors done!")

        self.w2v_model = model
        return model

```

```

def get_embeddings(self,):
    if self.w2v_model is None:
        print("model not train")
        return {}

    self._embeddings = {}
    for word in self.graph.nodes():
        self._embeddings[word] = self.w2v_model.wv[word]

    return self._embeddings

```

Run graph embedding

```

>]: G = nx.read_edgelist('lab1_big_edgelist.txt',create_using=nx.DiGraph(),nodetype=None,data=[('weight',int)])# Read graph
model = DeepWalk(G,walk_length=10,num_walks=80)#init model
model.train(window_size=5,iter=3)# train model
embeddings = model.get_embeddings()# get embedding vectors

Learning embedding vectors...
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
Learning embedding vectors done!

```